

IndieQA

An autonomous QA bot for indie studios. It plays your level ~240× faster than real time, hunts physics and collision bugs, and hands you every bug with where it happened and the exact buttons to reproduce it.

Collision bugs ship. Players find them on launch day.

Indie studios can't afford a QA team.

Walking through walls, falling out of the map, getting stuck: easy to miss by hand.

They surface as negative Steam reviews and refund requests.

Testers often can't say which inputs caused the bug, so reproducing it is slow.

HUMAN QA TESTER

\$25/h

and still misses physics edge cases

From autonomous play to a bug report you can replay.

01

Agent plays

An autonomous agent plays the level. No human at the controls.

buttons →

02

Engine simulates

Fixed 60 FPS physics, headless (no window), far faster than real time.

frame state →

03

Telemetry

Every frame: position, velocity, buttons, grounded, collision.

CSV →

04

Bug detector

Classifies physics anomalies. One report per bug: type, severity, location, last 30 inputs.

reports →

05

Dashboard

Local website: coverage heatmap, red bug markers, filterable log, export.

localhost

An autonomous, rule-based agent built to break geometry.

State machine · not machine learning · not an LLM

MODE 1

Boundary_Seeker

Senses the level with raycasts (wall ahead?) and ground probes (ledge ahead?). Walks to walls, edges and corners and hugs them, because that's where collision bugs live.

MODE 2

Input_Spammer

When touching a wall, fires rapid non-linear button combos (diagonal + jump, direction toggles) to force the physics into broken states.

Zero training data

works day one

Fully offline

runs on a laptop

Deterministic

same seed = same run

Explainable

every decision is a rule

Cheap

~\$0.04/h compute

We planted 3 bugs. The bot found them on its own.

PLANTED BUG	DETECTION SIGNATURE	HITS IN 10 MIN
Wall Clip 2-px seam in the right wall	> 25 px in one frame while colliding	22
Infinite Fall hole with no bottom, no reset	> 120 frames falling, touching nothing	48
Softlock Pit 150 px deep, jump is 100 px	buttons held > 300 frames, moved < 5 px	16
Out of Bounds	position outside the map	—



Test level • red seam = Wall Clip • red gap = Infinite Fall • yellow = Softlock Pit

Every bug, where it happened, and how to replay it.



Header game time tested, frames analysed, bugs found, cost banner	Map heatmap of where the bot explored, red marker at every bug	Bug log filter by severity / type, reproduction sequence per bug	Export bug reports as Markdown
--	---	---	--

All three bugs in under 15 seconds.

1 · Watch it play (real time, 60 FPS)

```
python main.py --seed 11
```

1.4s **3.4s** **12.4s**

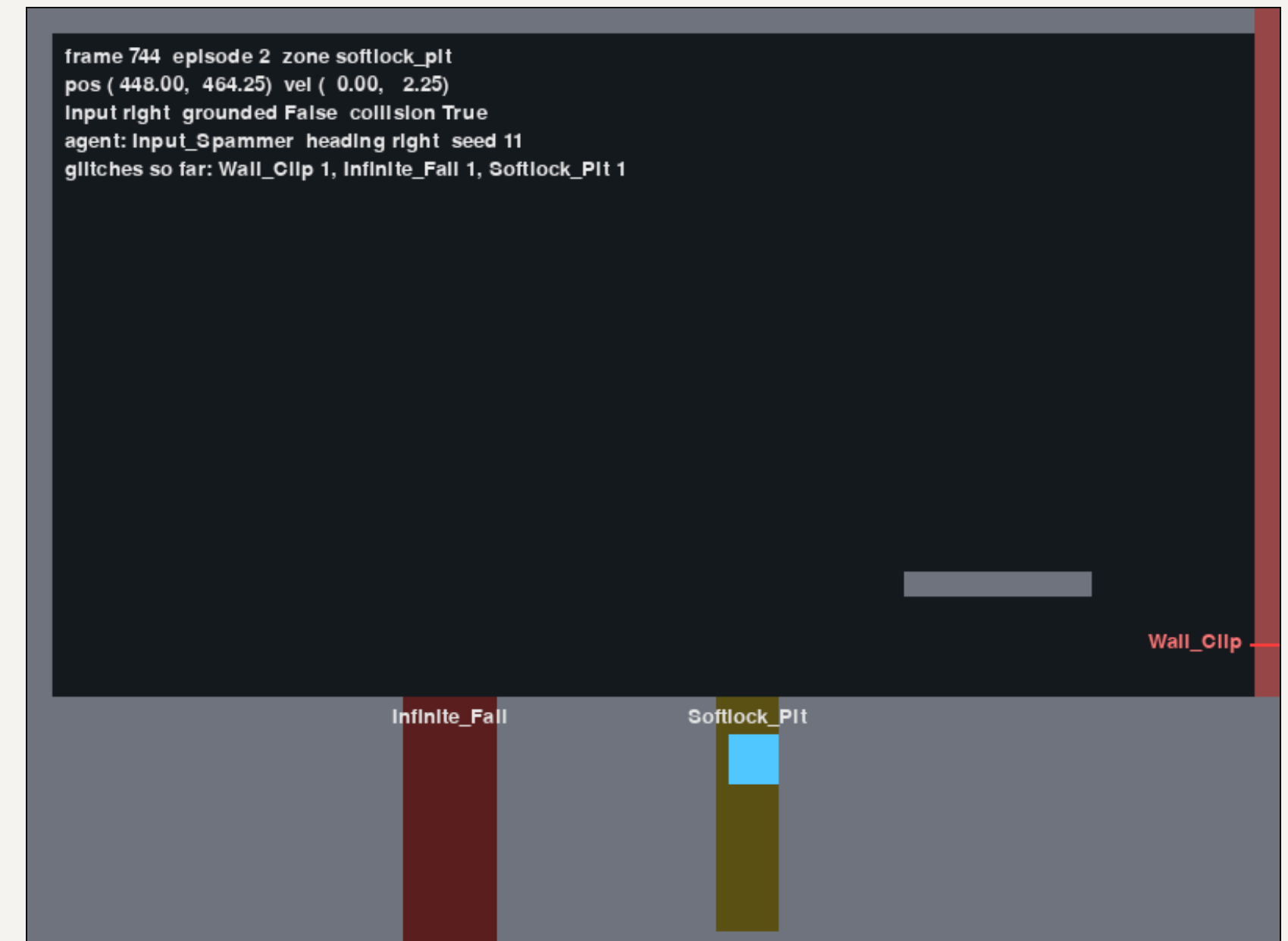
Wall Clip Infinite Fall Softlock

2 · 10 minutes of testing in ~3 seconds

```
python main.py --headless --frames 36000 --analyze
```

3 · Open the website: map, bug list, one bug's steps

```
streamlit run dashboard/app.py
```



Game window, seed 11, frame 744 (12.4 s): player stuck in the Softlock Pit, HUD shows all 3 glitches found

Find the bug before your players review it.

BEFORE LAUNCH

Collision bugs are caught on your machine, not in Steam reviews and refund requests.

EVERY BUILD

10 minutes of play in ~2.5 s means you can re-test the level after every change.

FIX FAST

Each bug ships with location + last 30 inputs. Same seed, same bug, 100% reproducible.

Three failure modes we state up front.

- 1

Cannot evaluate narrative or quest-sequencing logic.

It tests physics, not story.
- 2

High false-positive rate on non-standard physics with custom inertia curves.

Thresholds like 25 px/frame assume standard platformer physics.
- 3

Struggles with procedurally generated maps with no static geometry reference.

HOW WE VALIDATE OURSELVES

A planted-bug level + an automatic check of detector output vs the engine's ground truth.

found / missed / false alarms, per bug type

BUG	DETECTED
Wall Clip	22/22
Infinite Fall	[detector results]
Softlock	[detector results]

HUMAN QA

\$25/h

INDIEQA · LOCAL COMPUTE

\$0.04/h

Team estimate. And one IndieQA hour covers far more than one hour of play: headless runs ~240× real time.

NEEDS NOTHING

No training data · no cloud · no API keys
· no GPU. ~1 μs telemetry overhead
per frame.

BUILT TODAY

Python + Pygame engine, agent, telemetry,
detector, reporter, Streamlit dashboard.
One command.

UNITY & GODOT

ROADMAP

Plug-ins in C# and GDScript stream the
same telemetry format over WebSocket.
Same detector, same dashboard. *Not
built yet.*

Four people, one pipeline.

P1

Engine & agent

Physics engine, planted glitches, telemetry, autonomous agent.

[Name]

P2

Detection

Bug detector and per-bug reporter.

[Name]

P3

Pitch

Story, deck and demo.

[Name]

P4

Website

Streamlit dashboard: map, bug log, export.

[Name]

Let the bot find the bug **before** your players do.

NEXT · 1

Unity (C#) and Godot (GDScript) telemetry plug-ins over WebSocket.

NEXT · 2

Per-game detector thresholds, to cut false positives on custom physics.

NEXT · 3

Run it on real indie levels.